

PROJECT REPORT OF CO-OP Project at Industry (Module-2)
ON
On Demand Airflow Review Environments
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

**Yatin
Yash Goyal
Vidhan Gupta
Anmol Singla**

Supervised By:

**Mr. Rajat Takkar
Chitkara University**

2210992569, 2210992554, 2210992523, 2210931001



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
1	Acknowledgement	4
2	Abstract	5
3	Introduction	6
4	Methodology	6
5	Tools and Technologies	6-7
6	Implementation	7
7	Results	8
8	Conclusion and Future Scope	8
9	References	9

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “On Demand Airflow Review Environments” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Mr. Rajat Takkar. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Yatin

Date: ...29-April-2026.....

2210992569

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Mr. Rajat Takkar

Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our profound gratitude to our mentor, Mr. Rajat Takkar, for their invaluable guidance, continuous support, and motivation throughout the duration of this project. Their deep insights and constructive feedback were instrumental in the successful completion of this work.

ABSTRACT

In the current development structure, Airflow developers face significant hurdles as pushing tests and experiments into a single staging branch causes the environment to drift from the actual production setup over time. This divergence leads to manual, risky deployment processes, delayed release cycles, and breaks good version control habits. To address these bottlenecks, this project proposes an automated system that spins up a temporary, dedicated Airflow space every time a developer opens a Merge Request. Powered by GitLab CI/CD pipelines, the system leverages Kubernetes (EKS) for container orchestration, Helm for deployment configurations, Docker for containerization, and temporary PostgreSQL databases for metadata storage.

1. Introduction

- **Problem Background:**

- In the current structure, Airflow developers face a major issue because everyone pushes their tests and random experiments into one single staging branch. Over time, this causes the staging environment to look completely different from the actual production environment, creating a difficulty for developers trying to test their changes. When it is finally time to deploy, developers are forced to cherry-pick which code updates need to be pushed. Doing this manually is risky, can often lead to mistakes, breaks good version control habits, and can result in delayed release and production failure.

- **Objective:**

- The primary goal of this project is to fix these bottlenecks by spinning up a temporary, dedicated Airflow space every time a developer raises a Merge Request. By providing each branch with its own exact replica of the main setup, developers can test their work more accurately. This ensures that getting new features into production is much faster and efficient.

2. Methodology : -

The methodology for deploying this system involves rolling it out in three distinct steps:

1. Getting the basic CI/CD pipelines working.
2. Managing the full environment setup and the teardown process.
3. Tweaking all components for better performance and security.

3. Tools and Technologies : -

- **GitLab CI/CD:** Used as the trigger to run the automation for the entire process.
- **Kubernetes (EKS):** Utilized to host the containers and manage container orchestration.
- **Helm:** Used to manage complex configurations and handle deployment details to create "Review Apps".

- **Docker:** Used to pack everything into containers.
- **PostgreSQL:** Runs within Docker to serve as temporary databases that handle backend data and store metadata.

4. Implementation : -

- **System Overview:**

- The system automatically sets up separate review spaces for building Airflow DAGs, which allows the team to double-check their work before merging anything into the production environment. All parts combine so that every feature receives a fresh, working copy of the production environment.

- **Automation:**

- The system automatically builds a new environment the moment it detects a new Merge Request via GitLab CI/CD.

- **Isolation Architecture:**

- It uses individual namespaces (such as airflow-mr-<MR ID>) to keep things separated, alongside temporary PostgreSQL databases. Temporary databases are specifically used to stop test data from spilling over into other testing cycles and to keep cloud costs down.

- **Developer Access:**

- Once generated, developers receive a dedicated link and login details so they can get access to their specific Airflow UI.

- **Teardown Process:**

- Once a Merge Request is closed or merged, the system deletes everything—including namespaces, deployments, and databases—so nothing is left running. This leads to cost efficiency.

5. Results : -

- **Workflow Enhancements:**

- The project delivers a completely hands-off system that gives every Merge Request its own Airflow playground.

- **Increased Efficiency:**

- It speeds up coding because developers no longer have to wait in line to test their work.

- **Quality Assurance:**

- It cuts down on bugs since everything gets tested in total isolation.

- **Environment Stability:**

- It stops the staging branch from getting too cluttered and cuts down on the messy workaround of duplicating DAGs.

- **Cost Efficiency:**

- It saves money because it dynamically destroys the cloud resources as soon as the testing is done.

6. Conclusion And Future Scope : -

- This project successfully delivers a complete, automated way to handle short-lived Airflow setups. By connecting GitLab Review Apps with Kubernetes clusters, the system solves the persistent problem of staging environments drifting away from production, which is a common issue in big data teams.

Future Scope : -

- **Performance Optimization:** As part of the ongoing rollout strategy, a primary future objective is tweaking the entire automated setup to achieve better overall performance.
- **Security Enhancements:** Further refinement of the system will focus on implementing stronger security measures across the Kubernetes clusters, Helm configurations, and temporary Docker databases.

7. References : -

- <https://docs.aws.amazon.com/>
- <https://docs.gitlab.com/>